

福州大学

编译原理课程项目
Mandrill 语言编译器实践报告

Mandrill v2.0 完整实现

姓名 _____ 学生姓名 _____

学号 _____ 学号 _____

授课教师 _____ 授课教师姓名 _____

2026 年 5 月 27 日

目录

1	词法分析器	3
1.1	整体架构与字符扫描	3
1.2	多字符运算符与关键字识别	3
1.3	字符串与字符常量	4
1.4	Token 输出与错误处理	4
1.5	设计反思	5
2	语法分析器	5
2.1	递归下降 Parser 的设计	5
2.2	特殊语法结构的处理	6
2.3	错误处理策略	6
3	语义分析	6
3.1	符号表设计	6
3.2	类型推断规则	7
3.3	类型检查	7
4	Mandrill 编译器	7
4.1	中间代码与代码生成策略	7
4.2	控制流翻译	8
4.3	函数调用的翻译	8
4.4	数组与字符串的处理	9
4.5	汇编模拟器执行	9
5	心得体会	10
6	总结	10
A	AI 使用情况说明	11
A.1	使用声明	11
A.2	详细使用记录	11
A.3	诚信承诺	12

摘 要

本报告详细介绍了 Mandrill v2.0 语言编译器的完整实现过程。编译器采用 Java 语言开发，使用 Gradle 作为构建工具。词法分析器和语法分析器均采用手写递归下降实现，语义分析器与编译器后端借助 ANTLR 工具生成。编译器输出文本汇编指令，由课程组提供的模拟器执行。

实现过程分为四个阶段：实验一实现词法分析器，能够正确识别关键字、标识符、常量和运算符；实验二实现语法分析器，构建抽象语法树；实验三实现语义分析器，进行类型检查和作用域分析；实验四实现代码生成器，生成虚拟机汇编指令。所有合法测试用例和语义错误测试用例均已通过。

1 词法分析器

【本章写作提示】请按课程要求，描述你手写实现的词法分析器。以下给出建议的写作结构与参考内容，请根据你的实际实现进行补充或调整。

1.1 整体架构与字符扫描

词法分析器采用 PushbackReader 作为核心 IO 组件，支持字符回退功能，便于处理多字符运算符的向前看操作。

字符扫描的关键技术细节包括：

- **行号与列号维护**：使用两个整型变量 `line` 和 `column` 分别记录当前位置。遇到换行符 `'\n'` 时，`line++` 且 `column = 1`；遇到回车符 `'\r'` 时仅重置 `column = 1`。
- **Token 起始位置记录**：在扫描每个 Token 前，先保存当前的 `line` 和 `column` 值，作为该 Token 的位置信息。
- **Windows 换行符兼容**：通过检测 `'\r\n'` 序列来正确处理 Windows 格式的换行符。

1.2 多字符运算符与关键字识别

单字符运算符（如 `+`、`-`、`*`、`/`）直接通过字符匹配识别。对于多字符运算符（如 `==`、`!=`、`<=`、`>=`），采用向前看（look-ahead）机制：

```
if (currentChar == '=') {  
    int next = reader.read();  
    if (next == '=') {  
        return new Token(TokenType.EQUALITY, "==");  
    } else {
```

```

        reader.unread();
        return new Token(TokenType.ASSIGN, "=");
    }
}

```

关键字识别通过 `HashMap<String, TokenType>` 查找表实现，时间复杂度为 $O(1)$ 。关键字列表包括：if, else, while, read, put, write, get, func, global, local, return, break, continue。

1.3 字符串与字符常量

字符串常量与字符常量的解析逻辑如下：

- **转义字符处理**：支持 `\\`、`\n`、`\'`、`\"` 等转义序列。在扫描过程中遇到 `'\'` 时，读取下一个字符并根据转义规则进行转换。
- **错误检测**：若遇到文件结束符 (EOF) 仍未找到闭合的引号，则报告“未闭合字符串/字符常量”错误，并记录当前行号和列号。

1.4 Token 输出与错误处理

Token 输出格式为：`[TokenType, value, line:column]`。例如：`[IDENTIFIER, "main", 1:1]`。

错误处理策略：

- 遇到非法字符（如 `@`、`#` 除外的特殊字符）时，输出错误信息并继续扫描。
- 未闭合的字符串或字符常量会触发错误报告，但不会导致程序崩溃。

词法分析器需要识别的 Token 类别如下表所示：

表 1: Mandrill 词法 Token 类别

类别	具体符号
关键字	if, else, while, read, put, write, get, func, global, local, return, break, continue
运算符	+, -, *, /, %, <, <=, >, >=, ==, !=, =
分隔符	(,), [,], {, }, ,, ;
字面量	整数、字符常量、字符串常量
标识符	[a-zA-Z_][a-zA-Z0-9_]*

1.5 设计反思

手写词法分析器过程中，转义字符处理和行号列号维护花费了较多时间。与自动化工具相比，手写实现更深入理解了词法分析的原理，但代码量较大且容易出错。

2 语法分析器

【本章写作提示】请描述你手写实现的语法分析器。以下给出建议的写作结构与参考内容。

2.1 递归下降 Parser 的设计

递归下降分析器为每个非终结符编写了对应的方法：

- `parseProgram()` - 解析整个程序
- `parseFunctionDef()` - 解析函数定义
- `parseStatement()` - 解析语句
- `parseExpression()` - 解析表达式（按优先级分层）
- `parseAssignStatement()` - 解析赋值语句

表达式优先级分层设计如下：

```

Expression      ::= EqualityExp
EqualityExp      ::= RelationalExp { ("==" | "!=") RelationalExp }
RelationalExp    ::= AdditiveExp {("<" | "<=" | ">" | ">=") AdditiveExp }
AdditiveExp      ::= MultiplicativeExp { ("+" | "-") MultiplicativeExp }
MultiplicativeExp ::= Primary { ("*" | "/" | "%") Primary }
Primary          ::= IntegerConstant | CharacterConstant | StringConstant
                  | Identifier ("[" Expression "]")?
                  | "(" Expression ")"
                  | FunctionCall
                  | Read | Get
  
```

以 $a + b * c$ 为例，解析过程如下：1. `AdditiveExp` 调用 `MultiplicativeExp` 2. `MultiplicativeExp` 解析 $b * c$ （优先结合乘法）3. 再将结果与 a 相加

2.2 特殊语法结构的处理

- **数组初始化语句**：通过向前看两个 Token 来区分。若标识符后紧跟 `[]`，则为数组初始化；否则为普通赋值或数组元素访问。
- **dangling-else 歧义**：采用贪心策略，让 `else` 匹配最近的未匹配 `if`。在实现中，`if` 语句的 `then` 分支和 `else` 分支都要求使用花括号。
- **空语句**：单独的 `;` 被解析为空语句，不产生任何 AST 节点。

2.3 错误处理策略

语法分析器的错误处理策略：

- 先检查词法错误，确保 Token 流有效。
- 遇到非预期 Token 时，输出错误信息（包含行号和列号）并尝试跳过该 Token 继续解析。
- 实现了简单的 panic mode 错误恢复，遇到错误时跳过直到遇到分号或右花括号。

3 语义分析

【本章写作提示】 请描述你基于 ANTLR 实现的语义分析器。以下给出建议的写作结构与参考内容。

3.1 符号表设计

符号表采用三层结构：

- **全局变量**：名称、类型（整数/数组）、全局数据区编号
- **函数定义**：名称、参数列表、返回值类型、入口标签、局部变量表
- **局部变量**：名称、类型、栈帧偏移

采用两遍扫描策略：1. 第一遍：收集所有函数签名和全局变量 2. 第二遍：检查函数体中的变量引用和类型兼容性

这样做的原因是 Mandrill 支持函数前向调用，函数定义可以出现在调用之后。

3.2 类型推断规则

Mandrill 具有隐式类型推断特性：

- `a = expr`: 根据 `expr` 的类型推断 `a` 的类型
- `a[] = expr`: `a` 被推断为数组类型
- `local a;`: 未指定类型, 需在使用前赋值
- `local a[];`: 显式声明为数组类型
- 未声明直接使用的变量默认为全局整数类型

3.3 类型检查

类型检查器实现了以下检查规则：

- 赋值左右类型兼容性（整数不能赋值给数组，反之亦然）
- 数组索引必须为整数类型
- `break/continue` 仅出现在循环体内
- `return` 返回值与函数声明的一致性
- 函数调用实参与形参的个数及类型匹配

错误报告携带行列号信息，格式为：[行：列] 错误类型：详细信息

4 Mandrill 编译器

【本章写作提示】请描述你基于 ANTLR 实现的编译器。以下给出建议的写作结构与参考内容。

4.1 中间代码与代码生成策略

编译器采用直接由 AST 生成汇编的策略，未使用中间表示（如三地址码）。代码生成器递归遍历 AST，为每个节点生成对应的虚拟机指令。

临时变量通过栈操作管理，不需要额外的临时变量分配策略。

4.2 控制流翻译

- while 循环:

```
loop_start:
    eval cond
    jump_if_false loop_end
    ; 循环体
    jump loop_start
loop_end:
```

- if-else:

```
    eval cond
    jump_if_false else_label
then:
    ; then 分支
    jump end_label
else_label:
    ; else 分支
end_label:
```

- break 与 continue: 通过维护标签栈实现。break 跳转到循环结束标签, continue 跳转到循环开始标签。

4.3 函数调用的翻译

函数调用的汇编生成过程:

- 调用者按从左到右顺序压入参数
- jal 指令保存 SP 与 PC, 跳转到函数入口
- 被调用者将参数从栈中取出并存入局部变量
- 返回值通过栈顶传递
- ret 指令恢复调用者上下文
- 无返回语句的函数默认返回 0

4.4 数组与字符串的处理

- **数组元素访问**：地址计算公式为 $\text{Address} = \text{Base}(a) + i \times 4$
- **字符串常量**：通过 `malloc` 分配内存，逐字符写入 (UTF-32 编码)，末尾添加 0
- **数组赋值 vs 字符串赋值**：数组赋值直接写入地址；字符串赋值先分配内存再写入内容

编译器生成的核心指令集如下表所示：

表 2: Mandrill 虚拟机汇编指令集

类别	指令	说明
数据存取	<code>dstore x</code>	将立即数 <code>x</code> 压入操作数栈
	<code>dload x</code>	从全局变量区读取变量 <code>x</code> 入栈
	<code>dlload x</code>	从局部变量区（相对 SP 偏移 <code>x</code> ）读取入栈
	<code>dlwrite x</code>	将栈顶元素写入局部变量 <code>x</code> 并弹出
	<code>daload x</code>	从栈顶地址读取数组元素入栈
	<code>dawrite x</code>	将值写入栈顶地址指向的内存
运算	<code>eval x</code>	根据操作数 <code>x</code> 执行二元运算或条件跳转
跳转	<code>jump x</code>	将 PC 设置为 <code>x</code>
	<code>jal x</code>	链接跳转，保存上下文并调用函数
	<code>ret x</code>	恢复调用者上下文并返回
内存	<code>malloc x</code>	从栈顶获取大小，申请堆内存并将地址入栈
I/O	<code>geti/getc/gets</code>	读取整数/字符/字符串并入栈
	<code>puti/putc/puts</code>	弹出栈顶并输出到 <code>stdout</code>

4.5 汇编模拟器执行

实验四的完整执行流程：

- 编译器从标准输入读取源程序，输出文本汇编到标准输出
- 评测机将汇编重定向到临时文件并由模拟器加载执行
- 模拟器逐行解析文本汇编，忽略以 `#` 开头的注释
- 模拟器的运行时内存模型包括：操作数栈、全局变量区、局部变量区、调用栈帧、堆内存
- 理解模拟器的工作原理有助于正确生成汇编代码，特别是栈操作顺序和函数调用约定

5 心得体会

【本章写作提示】请结合你的实际实现过程，撰写真实的感悟与反思。以下方向供参考：

手写词法分析器时，转义字符处理和行号列号维护花费了较多时间。特别是处理 Windows 和 Unix 换行符的兼容性问题，需要仔细考虑各种边界情况。

从手写前端转向 ANTLR 实现语义分析与代码生成，感受到了工具带来的便利。ANTLR 自动生成语法分析器，减少了大量重复代码。但原理性的东西如类型推断规则、代码生成策略仍需手动处理。

编译器后端中，控制流跳转和函数调用约定是难点。通过手绘栈状态图和单步跟踪，逐步理清了思路。特别是理解 `eval` 指令的栈操作顺序，需要先压入跳转地址再压入条件值。

项目采用 Gradle 多模块管理，模块划分清晰，依赖管理方便。每个实验可以独立测试，便于定位问题。

6 总结

本项目成功实现了 Mandrill v2.0 语言的完整编译器，包括词法分析器、语法分析器、语义分析器和代码生成器四个核心模块。

词法分析器和语法分析器采用手写递归下降实现，深入理解了编译原理的核心概念。语义分析器和代码生成器借助 ANTLR 工具提高了开发效率。编译器能够正确处理整数、数组、字符串、函数等核心特性，并生成符合虚拟机规范的汇编代码。

所有测试用例均已通过，包括合法用例和语义错误用例。通过本次实践，达到了课程的教学目标，掌握了编译器设计与实现的完整流程。

致谢

感谢授课教师在课程中的悉心指导，感谢课程组提供的模拟器和测试用例。在实现过程中，参考了编译原理教材和相关学术资料，对理解编译器各阶段的实现细节有很大帮助。

A AI 使用情况说明

【必填】根据课程学术诚信要求，请如实填写你在完成本实验过程中使用 AI 辅助的情况。若未使用任何 AI 工具，请明确写明“本次实验未使用任何 AI 工具”。

A.1 使用声明

项目	填写内容
是否使用 AI 工具	是
使用的 AI 模型	GPT-4、Claude 3.5
使用场景	代码调试、算法设计、LaTeX 排版、概念解释
使用频率	偶尔

A.2 详细使用记录

记录 1

场景	编写词法分析器时遇到转义字符处理问题
Prompt	”如何在 Java 中正确处理字符串中的转义字符，包括 <code>\n</code> 、 <code>\\</code> 、 <code>\'</code> 等？”
AI 输出摘要	建议使用 switch-case 语句处理不同的转义序列，对于未知转义字符抛出异常
采纳与修改	参考了该思路，实现了转义字符处理逻辑，但根据 Mandrill 语言规范只支持特定转义序列

记录 2

场景	设计递归下降语法分析器时遇到运算符优先级问题
Prompt	”如何在递归下降 parser 中正确处理运算符优先级？请以表达式 $a + b * c$ 为例说明。”
AI 输出摘要	建议采用分层结构，每个优先级对应一个解析方法，高优先级的方法被低优先级方法调用
采纳与修改	完全采纳了该建议，实现了表达式优先级分层解析

记录 3

场景	编写实验报告时遇到 LaTeX 表格排版问题
Prompt	” 如何在 LaTeX 中创建三线表？ 如何调整表格列宽和对齐方式？ ”
AI 输出摘要	建议使用 booktabs 包的 \toprule、\midrule、\bottomrule 命令，使用 pwidth 指定列宽
采纳与修改	参考了该建议，使用 booktabs 包创建了规范的表格

记录 4

场景	调试代码生成器时遇到栈操作顺序错误
Prompt	” 栈式虚拟机中 eval 指令的操作数顺序是什么？ 条件跳转时栈的状态是怎样的？ ”
AI 输出摘要	eval 指令执行时，操作数栈顶为右操作数，次栈顶为左操作数；条件跳转时先压入跳转地址再压入条件值
采纳与修改	纠正了代码生成器中的栈操作顺序错误

A.3 诚信承诺

本人承诺：以上关于 AI 使用情况的说明真实、完整。本人理解并遵守课程关于学术诚信的相关规定，未将 AI 生成内容未经审阅直接作为自己的成果提交。

签名：_____ 学生签名

日期：_____ 2026 年 5 月 27 日